

Machine Learning - Normalizacion y LOGR

July 20, 2026



Autor: Elvis Pachacama **Recopilado por:** Pablo Aguilar * Repositorio: GitHub * URL: <https://github.com/Exosphere-Idyllic/Machine-Learning>

0.1 Práctica Académica

0.1.1 Normalización y Estandarización de Datos en Inteligencia Artificial

0.2 Análisis y aplicación de técnicas de normalización y estandarización en conjunto de datos para modelos de aprendizaje automático

Objetivo General Aplicar técnicas de normalización y estandarización de datos mediante herramientas de Python, con el fin de comprender su impacto en la preparación de datos y en el desempeño de modelos de aprendizaje automático.

Objetivos específicos

- Identificar diferencias de escala entre variables numéricas.
- Aplicar técnicas de normalización (Min-Max).
- Aplicar técnicas de estandarización (Z-score).
- Analizar la importancia del preprocesamiento en modelos de IA.

Antecedentes / Fundamentación teórica En el contexto del aprendizaje automático, los algoritmos suelen verse afectados por la escala de los datos. Variables con valores grandes pueden dominar el comportamiento del modelo, generando sesgos y resultados poco confiables. Para mitigar este problema, se emplean técnicas como:

- **Normalización:** transforma los datos a un rango específico (generalmente $[0,1]$).
- **Estandarización:** ajusta los datos para que tengan media 0 y desviación estándar 1.

Estas técnicas son fundamentales en algoritmos como:

- KNN
- Regresión logística
- Redes neuronales

Escenario de práctica Una empresa de análisis de datos desea desarrollar modelos predictivos utilizando información de clientes. Sin embargo, se ha identificado que las variables presentan diferentes escalas, lo que puede afectar el rendimiento de los modelos. Se solicita al analista aplicar técnicas de preprocesamiento y limpieza de datos para mejorar la calidad de los datos.

0.2.1 Conjunto de datos a utilizar

- Dataset 1 (Conceptual): Iris dataset
- Dataset 2 (aplicación real): California Housing Dataset

```
[3]: # Importación de librerías
import pandas as pd
import numpy as np
# from sklearn.datasets import load_iris
import zipfile
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn import preprocessing
```

```
[6]: # Configuramos el entorno de descarga
import os
os.environ['KAGGLE_CONFIG_DIR'] = "C:/Users/USER/.kaggle"
```

```
[7]: # Creamos la carpeta dataset
os.makedirs("dataset", exist_ok=True)
```

```
[8]: # Descargamos el dataset
!kaggle datasets download -d ucml/iris
```

Dataset URL: <https://www.kaggle.com/datasets/ucml/iris>

License(s): CC0-1.0

iris.zip: Skipping, found more recently modified local copy (use --force to force download)

```
[9]: # Descomprimos el archivo
with zipfile.ZipFile("iris.zip", 'r') as zip_ref:
    zip_ref.extractall("dataset")
```

```
[10]: # Cargamos el conjunto de datos
df = pd.read_csv("dataset/Iris.csv")
```

```
[11]: # Mostramos el conjunto de datos
df.head()
```

```
[11]:
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

```
[12]: # Eliminamos columnas innecesarias en la limpieza de datos. Como vemos en el
      ↪ dataframe,
      # la columna Id no aporta nada a nuestro modelo, por lo tanto la eliminamos.
      if 'Id' in df.columns:
          df = df.drop(columns=["Id"])
      df.head()
```

```
[12]:
```

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

```
[13]: # Mostramos los detalles de nuestro dataset
      df.info()

      # Verificamos si tenemos datos nulos; si los tenemos, los sumamos
      df.isnull().sum()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   SepalLengthCm    150 non-null   float64
1   SepalWidthCm     150 non-null   float64
2   PetalLengthCm    150 non-null   float64
3   PetalWidthCm     150 non-null   float64
4   Species          150 non-null   str
dtypes: float64(4), str(1)
memory usage: 6.0 KB
```

```
[13]: SepalLengthCm    0
      SepalWidthCm     0
      PetalLengthCm    0
      PetalWidthCm     0
      Species          0
      dtype: int64
```

```
[14]: # Separamos los datos de entrada de nuestra variable objetivo
X = df.drop(columns=['Species'])
y = df['Species']
```

```
[15]: # Exploramos / generamos un análisis exploratorio del conjunto de datos
X.describe()
```

```
[15]:
```

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

0.2.2 Conclusión

Cuando observamos la tabla, lo primero que se observa es que cada variable tiene valores diferentes en tamaño y rango. Por ejemplo:

- **PetalLengthCm** tiene valores desde 1.0 hasta 6.9, lo que significa que varía bastante.
- En cambio, **SepalWidthCm** va aproximadamente de 2.0 a 4.4, es decir, tiene menos variación.

Esto nos dice que no todas las variables están en la misma escala.

¿Qué significa? Al ver los datos, vemos que algunas variables tienen números más grandes que otras, y en modelos como KNN:

- Los números grandes “pesan más”.
- Influyen más en el resultado.

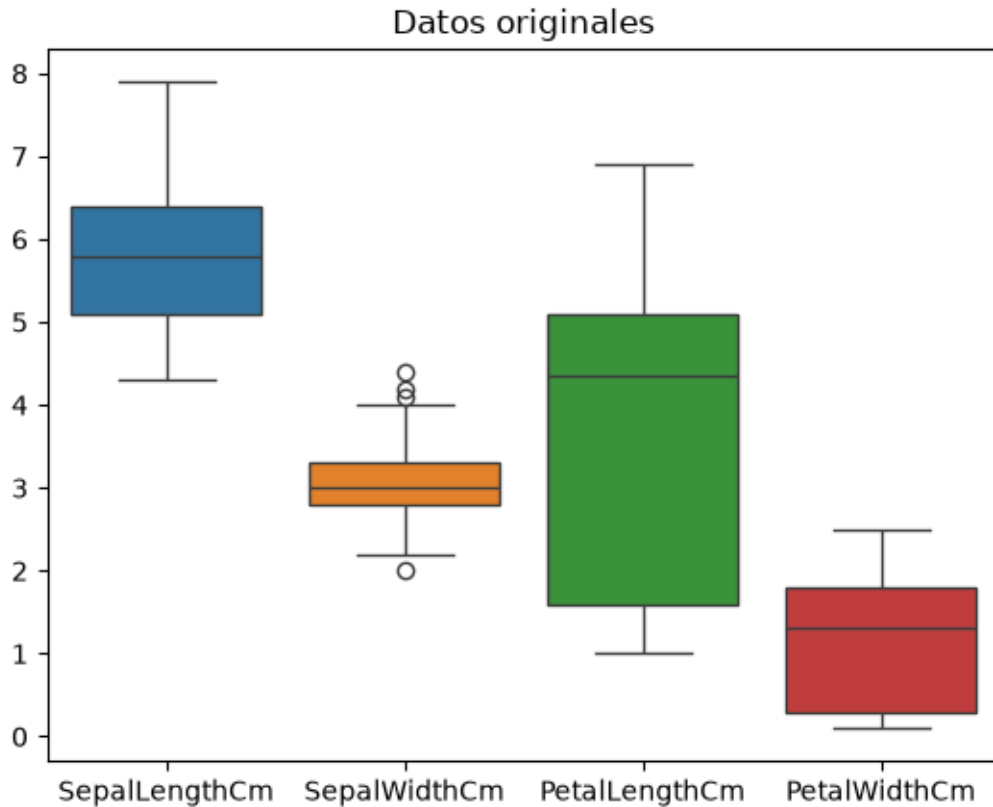
Si no hacemos nada y trabajamos con estos datos, el modelo va a tomar decisiones basadas principalmente en el atributo **PetalLengthCm** y menos en las demás variables. Esto no es conveniente, y a esto lo llamamos **sesgo**.

Para no tener sesgo dentro de nuestros datos, lo que hacemos es:

- Normalización
- Estandarización

Esto permite que las variables estén en la misma escala y tengan la misma importancia.

```
[16]: # Dibujamos los datos originales
sns.boxplot(data=X)
# Ponemos un título a la imagen
plt.title("Datos originales")
plt.show()
```



0.2.3 Interpretación de la imagen

Este gráfico nos permite observar que los datos no están en la misma escala y que algunas variables tienen mayor variabilidad que otras. Esto puede afectar el comportamiento de los modelos de inteligencia artificial, por lo que es necesario aplicar técnicas como la normalización o estandarización para equilibrar la influencia de todas las variables.

```
[17]: # Normalizamos el conjunto de datos
normalizer = preprocessing.MinMaxScaler()
X_norm = normalizer.fit_transform(X)
X_norm = pd.DataFrame(X_norm, columns=X.columns)
X_norm.head()
```

```
[17]:   SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm
0         0.222222         0.625000         0.067797         0.041667
1         0.166667         0.416667         0.067797         0.041667
2         0.111111         0.500000         0.050847         0.041667
3         0.083333         0.458333         0.084746         0.041667
4         0.194444         0.666667         0.067797         0.041667
```

```
[18]: X_norm.describe()
```

```
[18]:
```

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	150.000000	150.000000	150.000000
mean	0.428704	0.439167	0.467571	0.457778
std	0.230018	0.180664	0.299054	0.317984
min	0.000000	0.000000	0.000000	0.000000
25%	0.222222	0.333333	0.101695	0.083333
50%	0.416667	0.416667	0.567797	0.500000
75%	0.583333	0.541667	0.694915	0.708333
max	1.000000	1.000000	1.000000	1.000000

```
[19]: # Estandarización
standarizer = preprocessing.StandardScaler()
# Estandarizamos los datos
X_std = standarizer.fit_transform(X)
X_std = pd.DataFrame(X_std, columns=X.columns)
X_std.head()
```

```
[19]:
```

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
0	-0.900681	1.032057	-1.341272	-1.312977
1	-1.143017	-0.124958	-1.341272	-1.312977
2	-1.385353	0.337848	-1.398138	-1.312977
3	-1.506521	0.106445	-1.284407	-1.312977
4	-1.021849	1.263460	-1.341272	-1.312977

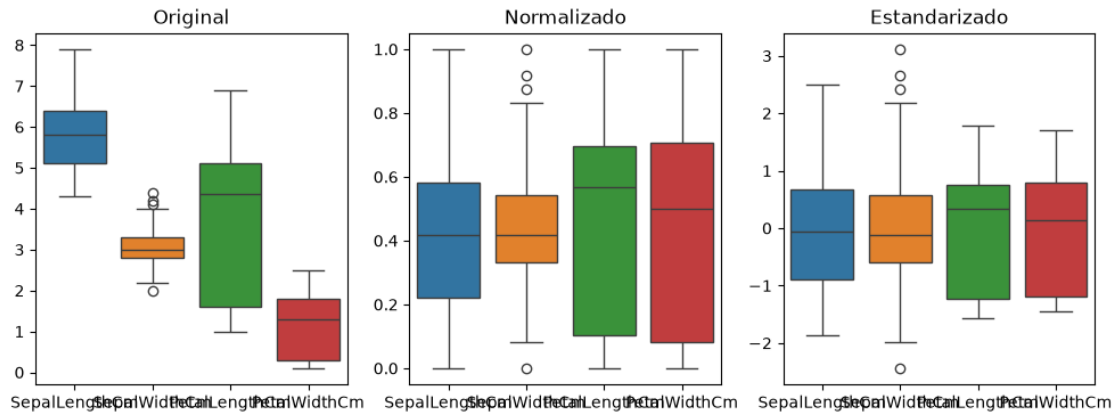
```
[20]: # Comparación visual
plt.figure(figsize=(12,4))

plt.subplot(1,3,1)
sns.boxplot(data=X)
plt.title("Original")

plt.subplot(1,3,2)
sns.boxplot(data=X_norm)
plt.title("Normalizado")

plt.subplot(1,3,3)
sns.boxplot(data=X_std)
plt.title("Estandarizado")

plt.show()
```



[]:

1 Programa15.Clasificacion.LOGR

1.1 PAO25-25 - CLASIFICACIÓN (Regresión Logística)

Elvis Pachacama

```
[22]: import numpy as np
import matplotlib.pyplot as plt
import sklearn.metrics as metrics
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_predict, cross_val_score,
    train_test_split, KFold
from sklearn import preprocessing
from evaluacion_funciones import *
```

```
-----
ModuleNotFoundError                                Traceback (most recent call last)
Cell In[22], line 8
      4 from sklearn.datasets import load_iris
      5 from sklearn.linear_model import LogisticRegression
      6 from sklearn.model_selection import cross_val_predict, cross_val_score,
    ↪ train_test_split, KFold
      7 from sklearn import preprocessing
----> 8 from evaluacion_funciones import *

ModuleNotFoundError: No module named 'evaluacion_funciones'
```

```
[23]: # Carga de datos.
datos = load_iris()
X = datos.data[:,2:] # Utilizamos solo 2 atributos.
y = datos.target
print(np.shape(X))
```

(150, 2)

```
[24]: # Métricas de evaluación.
metricas = {
    'ACC': metrics.accuracy_score,
    'PREC': lambda y_true, y_pred:
        metrics.precision_score(y_true, y_pred, average='micro'),
    'RECALL': lambda y_true, y_pred:
        metrics.recall_score(y_true, y_pred, average='micro'),
    'F1': lambda y_true, y_pred:
        metrics.f1_score(y_true, y_pred, average='micro'),
}
```

```
[25]: # 1) Partición de datos externa
X_training, X_testing, y_training, y_testing = train_test_split(X, y,
    ↪test_size=0.2, random_state=42)
print(np.shape(X_training))
```

(120, 2)

1.1.1 ———- TRAINING ———-

```
[26]: # 2) Extracción de características
```

```
[27]: # 3) Estandarización de los datos de entrenamiento
standardizer = preprocessing.StandardScaler()
X_std = standardizer.fit_transform(X_training)
# print(X_std)
```

```
[28]: # 4) Selección de atributos
```

```
[29]: # 5) Construcción del algoritmo de aprendizaje.
algoritmos = {'LOGR': LogisticRegression(penalty='l1', solver='saga',
    ↪max_iter=1000,
                                     random_state=42, multi_class='ovr')}
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[29], line 2
      1 # 5) Construcción del algoritmo de aprendizaje.
----> 2 algoritmos = {'LOGR': LogisticRegression(penalty='l1', solver='saga',
    ↪max_iter=1000,
```



```

3 random_state=42,
↳multi_class='ovr'})

TypeError: LogisticRegression.__init__() got an unexpected keyword argument
↳'multi_class'

```

```

[30]: # 5.1) Validación cruzada interna y Optimización de los hiperparámetros
y_pred = {}
for nombre, alg in algoritmos.items():
    y_pred[nombre] = cross_val_predict(alg, X_std, y_training,
                                       cv=KFold(n_splits=10, shuffle=True,
↳random_state=42))
    results = evaluacion(y_training, y_pred[nombre], metricas)
    print(metrics.confusion_matrix(y_training, y_pred[nombre]))
    print(results)
    # results = cross_val_score(alg, X_std, y_training, cv=KFold(n_splits=10,
↳shuffle=True, random_state=42))
    # print("Accuracy: %0.4f +/- %0.4f" % (results.mean(), results.std()))

```

```

-----
NameError                                Traceback (most recent call last)
Cell In[30], line 3
      1 # 5.1) Validación cruzada interna y Optimización de los hiperparámetros
      2 y_pred = {}
----> 3 for nombre, alg in algoritmos.items():
      4     y_pred[nombre] = cross_val_predict(alg, X_std, y_training,
      5                                       cv=KFold(n_splits=10,
↳shuffle=True, random_state=42))
      6     results = evaluacion(y_training, y_pred[nombre], metricas)

NameError: name 'algoritmos' is not defined

```

```

[31]: # 5.2) Entrenamiento del modelo definitivo
model = algoritmos['LOGR'].fit(X_std, y_training)

# Visualización de las fronteras de decisión
mapa_modelo_clasif_2d(X_std, y_training, model, results, nombre)

```

```

-----
NameError                                Traceback (most recent call last)
Cell In[31], line 2
      1 # 5.2) Entrenamiento del modelo definitivo
----> 2 model = algoritmos['LOGR'].fit(X_std, y_training)
      3
      4 # Visualización de las fronteras de decisión

```

```
5 mapa_modelo_clasif_2d(X_std, y_training, model, results, nombre)
```

```
NameError: name 'algoritmos' is not defined
```

1.1.2 ———- PREDICTION ———-

```
[32]: # 6) Extracción de las características de test
```

```
[33]: # 7) Estandarización de las características de test
X_test_std = standardizer.transform(X_testing)
```

```
[34]: # 8) Selección de los atributos de test
```

```
[35]: # 9) Predicción del conjunto de test
y_pred_test = model.predict(X_test_std)
print(y_pred_test)
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[35], line 2
      1 # 9) Predicción del conjunto de test
----> 2 y_pred_test = model.predict(X_test_std)
      3 print(y_pred_test)

NameError: name 'model' is not defined
```

```
[36]: # 10) Evaluación del modelo sobre el conjunto de test
results = evaluacion(y_testing, y_pred_test, metricas)
print(results)
print(metrics.confusion_matrix(y_testing, y_pred_test))
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[36], line 2
      1 # 10) Evaluación del modelo sobre el conjunto de test
----> 2 results = evaluacion(y_testing, y_pred_test, metricas)
      3 print(results)
      4 print(metrics.confusion_matrix(y_testing, y_pred_test))

NameError: name 'evaluacion' is not defined
```

```
[37]: # Ploteamos la curva ROC
y_proba_test = model.predict_proba(X_test_std) # "predict_proba" para extraer
↳ probabilidades en vez de predicciones
```

```

y_test_bin = preprocessing.label_binarize(y_testing, classes=[0,1,2]) # Usar
↳ "label_binarize" en el caso de problemas multiclase
auc = metrics.roc_auc_score(y_testing, y_proba_test, multi_class='ovr') # Area
↳ Under the ROC curve (AUC)
fpr, tpr, th = metrics.roc_curve(y_test_bin[:,1], y_proba_test[:,1])

plt.plot(fpr, tpr)
plt.xlabel('False positive rate')
plt.ylabel('True positive rate')
plt.title('AUC = ' + str(np.round(auc,4)))
plt.grid()
plt.show()

```

```

-----
NameError                                Traceback (most recent call last)
Cell In[37], line 2
      1 # Ploteamos la curva ROC
----> 2 y_proba_test = model.predict_proba(X_test_std) # "predict_proba" para
↳ extraer probabilidades en vez de predicciones
      3 y_test_bin = preprocessing.label_binarize(y_testing, classes=[0,1,2])
↳ Usar "label_binarize" en el caso de problemas multiclase
      4 auc = metrics.roc_auc_score(y_testing, y_proba_test, multi_class='ovr')
↳ # Area Under the ROC curve (AUC)
      5 fpr, tpr, th = metrics.roc_curve(y_test_bin[:,1], y_proba_test[:,1])

NameError: name 'model' is not defined

```

[]: